MySQL & PostgreSQL — Complete Cheat Sheet

Roman Urdu Training Guide · Basic → Advance · Practical Tasks ke saath · Full-Stack Developer Track

> **Kaise parhna hai:** Part 0 se shuru karo (foundation), phir MySQL aur PostgreSQL ki apni-apni cheat sheet (dono ka core SQL same hai — darasal SQL ek hi language hai, difference sirf kuch commands aur features mein hai). Part 4 differences samjhega, phir Part 5 ke practical tasks banao — **yahi asal seekhne ka zariya hai**. Part 6 ka 30-day plan follow karo. Ek hi project (e-commerce) dono databases mein banao — differences khud clear ho jayenge.

Part 0 — Foundation: Pehle ye 6 cheezein samjho

Term	Easy explanation
Database	Data ka organized store — jaise Excel ki bahut si sheets jo aapas mein judi hon.
RDBMS	Relational Database Management System — software jo data ko *tables* (rows + columns) mein rakhta hai aur tables ko aapas mein *relation* (key) se jorta hai. MySQL aur PostgreSQL dono RDBMS hain.
SQL	Structured Query Language — data se baat karne ki language. **90% SQL dono mein bilkul same hai.** Difference sirf kuch commands, data types aur extra features mein hai.
MySQL	Sabse zyada popular + simple RDBMS. Shared hosting aur web apps (LAMP stack) ka default. Ab Oracle ke paas.
PostgreSQL	Zyada powerful, strictly-standard, "database purists" ka favorite. Complex queries, data integrity aur advanced features (JSONB, window functions, schemas) mein strong.
Client	CLI ya GUI jisse database se connect hote ho. MySQL: `mysql` CLI / Workbench / DBeaver. PostgreSQL: `psql` CLI / pgAdmin / DBeaver.

Easy Way — bina install kiye abhi practice karo

> **Step 1:** Browser mein jao — `onecompiler.com/mysql` aur `onecompiler.com/postgresql` — turant dono chal jayenge, koi install nahi.
> **Step 2:** 3–4 din aise karo, phir local install karo: MySQL → mysql.com installer (Windows) ya `sudo apt install mysql-server` (Linux). PostgreSQL → postgresql.org installer ya `sudo apt install postgresql`.
> **Step 3 (recommended):** GUI ke liye **DBeaver** (free) — dono databases ek hi jagah manage hoti hain.

Basic concepts jo har DB mein same hain

- **Table** = data ki row/column wali structure. Har table ka ek **Primary Key (PK)** hota hai jo har row ko uniquely pehchanta hai (usually `id`).
- **Foreign Key (FK)** = doosri table ki PK ka reference — isi se "relation" banta hai. (e.g. `orders.student_id → students.id`)
- **CRUD** = Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE) — 4 kaam jo har app karta hai.
- **ACID** = Transaction ki 4 guarantees: *Atomicity* (ya to poora ya kuch nahi), *Consistency*, *Isolation*, *Durability*. Bank transfer ka classic example.
- **Index** = book ka index jaisa — WHERE/SEARCH fast karne ka pointer. Zyada index = fast SELECT but slow INSERT/UPDATE.

Part 1 — MySQL Cheat Sheet

1.1 Connect aur Client Commands (CLI mein)

```
mysql -u root -p                -- password poochega, phir enter
mysql -h localhost -u root -p   -- explicit host

SHOW DATABASES;                 -- saari databases list
CREATE DATABASE school;         -- nayi database banao
USE school;                     -- current database select karo
SHOW TABLES;                    -- tables list
DESC students;                  -- table ka structure (columns)
SHOW CREATE TABLE students;     -- exact CREATE statement
EXIT;                           -- bahar (ya: quit / \q)
```

1.2 DDL — Table banana aur structure badalna

```
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,   -- MySQL ka auto-increment
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  age TINYINT UNSIGNED,
  fee DECIMAL(10,2) DEFAULT 0,
  gender ENUM('male','female') DEFAULT 'male',
  is_active BOOL DEFAULT TRUE,        -- TINYINT(1) ki tarah chalta hai
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE students ADD COLUMN phone VARCHAR(20);
ALTER TABLE students DROP COLUMN phone;
ALTER TABLE students MODIFY age SMALLINT;      -- column type change
RENAME TABLE students TO learners;             -- table rename
TRUNCATE TABLE students;                       -- saara data + reset auto-inc
DROP TABLE students;
DROP DATABASE school;
```

1.3 CRUD — INSERT / SELECT / UPDATE / DELETE

```
-- INSERT
INSERT INTO students (name, email, age, fee)
VALUES ('Ali','ali@gmail.com', 20, 1500.50),
       ('Sara','sara@gmail.com', 22, 2000);

-- SELECT
SELECT * FROM students;
SELECT name, age FROM students WHERE age >= 18;
SELECT * FROM students WHERE name LIKE 'A%';          -- A se shuru
SELECT * FROM students WHERE dept_id IS NULL;
SELECT DISTINCT city FROM students;
SELECT * FROM students ORDER BY age DESC LIMIT 10 OFFSET 5;

-- UPDATE (hamesha WHERE check kar lo!)
UPDATE students SET fee = fee + 500 WHERE id = 3;

-- DELETE
DELETE FROM students WHERE id = 10;

-- Aggregates
SELECT COUNT(*) AS total, AVG(age), MIN(age), MAX(age), SUM(fee) FROM students;
```

1.4 JOIN, GROUP BY, Subquery, View, Index, Transaction

```
-- JOIN (3 alag tarah ke)
SELECT s.name, d.title
FROM students s
INNER JOIN departments d ON d.id = s.dept_id;   -- sirf match wale

SELECT s.name, d.title
FROM students s
LEFT JOIN departments d ON d.id = s.dept_id;    -- sab students, dept na ho to NULL

-- GROUP BY + HAVING (HAVING = groups par WHERE)
SELECT dept_id, COUNT(*) AS cnt, AVG(fee) AS avg_fee
FROM students
GROUP BY dept_id
HAVING COUNT(*) > 5;

-- Subquery
SELECT * FROM students WHERE age > (SELECT AVG(age) FROM students);

-- View (saved query)
CREATE VIEW active_students AS SELECT * FROM students WHERE is_active = 1;
SELECT * FROM active_students;

-- Index (search fast)
CREATE INDEX idx_students_email ON students(email);
SHOW INDEX FROM students;

-- Transaction (ACID demo)
START TRANSACTION;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
COMMIT;      -- save   (ghalti ho to: ROLLBACK;)
```

1.5 MySQL ke handy functions aur features

Kaam	MySQL syntax
Current date-time	`NOW()`, `CURDATE()`, `CURTIME()`
String join	`CONCAT(first,' ',last)`
Null-safe value	`IFNULL(city,'N/A')`
Random number	`RAND()`
Group mein string join	`GROUP_CONCAT(name SEPARATOR ', ')`
Date format	`DATE_FORMAT(created_at, '%d-%m-%Y')`
Substring	`SUBSTRING(name, 1, 5)`
Upsert (insert ya update)	`INSERT ... ON DUPLICATE KEY UPDATE col = VALUES(col);`
Pagination	`LIMIT 10 OFFSET 20`
Case-insensitive search	`LIKE` (default collation_ci ki wajah se)
Auto increment	`id INT AUTO_INCREMENT PRIMARY KEY`
JSON data	`JSON_EXTRACT(col, '$.name')` ya shorthand `col->'$.name'` (MySQL 5.7+)
Show running queries	`SHOW PROCESSLIST;`
Storage engines	InnoDB (default — transactions/FK support), MyISAM (purana, no transaction)

1.6 MySQL Backup & Restore (terminal se)

```
mysqldump -u root -p school > school_backup.sql        -- backup
mysql -u root -p school < school_backup.sql            -- restore
mysql -u root -p -e "SELECT * FROM students;" school    -- bina CLI andar gaye query
```

Part 2 — PostgreSQL Cheat Sheet

2.1 Connect aur psql Meta-Commands (ye sab backslash wale hain)

```
psql -U postgres                  -- superuser se connect (password mangega)
createdb school                   -- terminal se nayi database (ya SQL: CREATE DATABASE school;)
psql -U postgres -d school        -- specific database se connect

-- psql k andar ye meta commands chalti hain:
\l                -- databases ki list
\c school         -- database switch (connect)
\dt               -- tables list (d = describe, dt = tables)
\d students       -- table ka structure (columns + keys + indexes)
\d                -- current table kaunsa hai
\du               -- users/roles list
\dn               -- schemas list
\sf function_name -- function ka source
\?                -- saari commands ki help
\q                -- quit
```

> **Yaad rakho:** MySQL mein `SHOW TABLES;` — PostgreSQL mein `\dt` (SQL statement nahi, psql client command hai). PG ka GUI **pgAdmin** bhi yehi sab buttons se karta hai.

2.2 DDL — Table banana

```
-- Purana/popular tareeqa (SERIAL):
CREATE TABLE students (
  id SERIAL PRIMARY KEY,              -- auto-increment = SERIAL/BIGSERIAL
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  age SMALLINT,
  fee NUMERIC(10,2) DEFAULT 0,        -- MySQL DECIMAL = PG NUMERIC
  gender VARCHAR(10) DEFAULT 'male',  -- PG mein ENUM bhi banta hai (CREATE TYPE)
  is_active BOOLEAN DEFAULT TRUE,     -- PG ka native boolean (true/false)
  created_at TIMESTAMP DEFAULT NOW()
);

-- Modern SQL-standard tareeqa (PG 10+ recommended):
-- id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

ALTER TABLE students ADD COLUMN phone VARCHAR(20);
ALTER TABLE students DROP COLUMN phone;
ALTER TABLE students RENAME TO learners;
TRUNCATE TABLE students RESTART IDENTITY;   -- data + id counter reset
DROP TABLE students;
DROP DATABASE school;                        -- psql k bahar ya \c postgres pehle
```

2.3 CRUD (MySQL jaisa hi hai — yehi SQL hai)

```
INSERT INTO students (name, email, age, fee)
VALUES ('Ali','ali@gmail.com', 20, 1500.50),
       ('Sara','sara@gmail.com', 22, 2000);

SELECT * FROM students WHERE age >= 18 ORDER BY name LIMIT 10 OFFSET 5;
SELECT * FROM students WHERE name ILIKE 'A%';    -- PG: ILIKE = case-insensitive
UPDATE students SET fee = fee + 500 WHERE id = 3;
DELETE FROM students WHERE id = 10;
SELECT COUNT(*), AVG(age) FROM students;
```

2.4 JOIN, GROUP BY, View, Index, Transaction

```
-- JOIN / GROUP BY / HAVING — bilkul same syntax jaise MySQL
SELECT d.title, COUNT(s.id) AS cnt
FROM departments d
LEFT JOIN students s ON s.dept_id = d.id
GROUP BY d.title
HAVING COUNT(s.id) > 5;

-- View
CREATE VIEW active_students AS SELECT * FROM students WHERE is_active = TRUE;

-- Partial index (ye PG ka special feature hai — sirf active rows par index)
CREATE INDEX idx_students_email ON students(email);
CREATE INDEX idx_active ON students(id) WHERE is_active = TRUE;

-- Transaction (BEGIN bhi likh sakte ho)
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
COMMIT;      -- ya ROLLBACK;
```

2.5 PostgreSQL ke handy functions aur features

Kaam	PostgreSQL syntax
Current date-time	`NOW()`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`
String join	`first || ' ' || last` (ya `CONCAT(...)`)
Null-safe value	`COALESCE(city, 'N/A')`
Random number	`RANDOM()`
Group mein string join	`STRING_AGG(name, ', ')`
Date format	`TO_CHAR(created_at, 'DD-MM-YYYY')`
Substring	`SUBSTRING(name FROM 1 FOR 5)` (ya `SUBSTR(name,1,5)`)
Upsert (insert ya update)	`INSERT ... ON CONFLICT (id) DO UPDATE SET col = EXCLUDED.col;`
Case-insensitive search	`ILIKE` (LIKE PG mein case-sensitive hai)
Auto increment	`SERIAL` / `BIGSERIAL` ya `GENERATED AS IDENTITY`
JSON data	`JSONB` type + `col->>'name'`, `col @> '{"k":"v"}'` — indexed, bahut fast (JSON_EXTRACT se better)
Array column	`TEXT[]` — SQL arrays native (MySQL mein nahi)
Schema (database k andar folder)	`CREATE SCHEMA hr; CREATE TABLE hr.students...;`

Table inheritance	`CREATE TABLE child () INHERITS (parent);`
Query plan dekhna	`EXPLAIN ANALYZE SELECT ...;`
Extensions	`CREATE EXTENSION postgis;` (geo), pg_trgm (fuzzy search) etc.

2.6 PostgreSQL Backup & Restore

```
pg_dump -U postgres school > school_backup.sql      -- backup (custom format: -Fc)
psql -U postgres -d school -f school_backup.sql     -- restore
pg_dumpall -U postgres > all_databases.sql          -- saari DB backup
```

Part 3 — Common SQL: Dono par 100% same chalne wala core

Ye SQL hai — koi farq nahi. Interview aur daily kaam mostly isi par hai.

```
-- 1) Filtering
WHERE age BETWEEN 18 AND 25
WHERE city IN ('Karachi','Lahore')
WHERE name LIKE '%ah%'
WHERE email IS NOT NULL

-- 2) Sorting + Pagination
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;

-- 3) Joins ka types
INNER JOIN   -- sirf dono mein match
LEFT JOIN    -- left table ki sab rows (match na ho to NULL)
RIGHT JOIN   -- right table ki sab rows
FULL JOIN    -- dono ki sab rows (PG mein FULL OUTER JOIN; MySQL 8 mein bhi hai)
CROSS JOIN   -- har pair (cartesian)
SELF JOIN    -- ek table apne aap se (manager-employee)

-- 4) Set operations
SELECT name FROM a UNION SELECT name FROM b;        -- unique merge
UNION ALL;                                           -- duplicate ke saath
EXCEPT / MINUS;                                      -- a mein se b hatado
INTERSECT;                                           -- dono mein jo common

-- 5) Aliases — table ka naam chhota, column ka naam readable
SELECT s.name AS student, d.title AS department
FROM students AS s
JOIN departments AS d ON d.id = s.dept_id;

-- 6) CASE — SQL ka if-else
SELECT name,
  CASE WHEN fee > 2000 THEN 'Premium'
       WHEN fee > 1000 THEN 'Standard'
       ELSE 'Basic' END AS plan
FROM students;

-- 7) EXISTS (subquery se behtar performance kabhi kabhi)
SELECT * FROM departments d
WHERE EXISTS (SELECT 1 FROM students s WHERE s.dept_id = d.id);

-- 8) Constraints (dono same)
PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK (age >= 16)
```

Normalization (Interview + design ka sawaal) — ye kisi ek DB ka nahi, SQL concepts hain

Form	Rule	Misal (problem → solution)
1NF	Har cell mein single value — no lists, no repeats	`courses: "Math,Physics"` → alag `enrollments` table ya alag rows
2NF	1NF + har non-key column poore PK par depend kare (partial dependency hatao)	Composite PK (order_id, product_id) + product_name → product_name alag products table mein jaye
3NF	2NF + non-key columns aapas mein na depend karein (transitive dependency hatao)	`students` mein `dept_name` → alag `departments` table + FK

Part 4 — MySQL vs PostgreSQL: Differences (ye yaad karo)

Point	MySQL	PostgreSQL
Developer / Owner	Oracle Corporation	PostgreSQL Global Development Group (open community)
Language / Type	C/C++ · RDBMS	C · Object-Relational DBMS (zyada features)
Philosophy	Speed + simplicity ("fast aur easy")	Standard + correctness + power ("sahi aur powerful")
Auto increment	`AUTO_INCREMENT`	`SERIAL` / `BIGSERIAL` / `GENERATED AS IDENTITY`
Boolean	`BOOL` = TINYINT(1); 1/0	Native `BOOLEAN`; TRUE/FALSE
Identifier quoting	Backticks: `` `order` ``	Double quotes: `"order"`
JSON	`JSON` type (5.7+) — extra generated columns chahiye index k liye	`JSON` + **`JSONB`** (binary, indexable, operators `->>` `@>`) — bahut strong
Upsert	`ON DUPLICATE KEY UPDATE`	`ON CONFLICT ... DO UPDATE`
LIKE	Default case-insensitive (ci collation)	LAT-sensitive; `ILIKE` for insensitive
Random / Concat / Null	`RAND()` · `CONCAT()` · `IFNULL()`	`RANDOM()` · `||` · `COALESCE()`
String aggregate	`GROUP_CONCAT()`	`STRING_AGG()`
Database vs Schema	Database = schema (flat)	Database → **Schema** → table (ek DB mein kai schemas, e.g. `public`)
Window functions	8.0+ mein available	Pehle se available (7.4+) aur mature
Advanced features	Straightforward, kam surprises	Table inheritance, exclusion constraints, custom types, arrays, materialized views, partial indexes, extensions (PostGIS)
Concurrency	InnoDB row locking + MVCC	Full MVCC — long transactions par readers block nahi hote
Case sensitivity (DB naam)	Linux par case-sensitive; Windows nahi	Hamesha lowercase (bina quotes ke)
Replication / Ecosystem	Bohat common shared hosting, cPanel default	Managed hosting (Neon, Supabase, RDS) + enterprise (EDB)
Best for	Web apps, blogging/e-commerce CMS, read-heavy simple workloads, hosting availability	Complex queries, analytics, strict data integrity, geospatial (PostGIS), JSON-heavy apps, startups (Supabase)
Full-Stack context	classic PHP/Laravel apps, WAMP/XAMPP	Next.js + Supabase, Django default, Rails default

> **Trainer tip:** "Kaunsa behtar?" ka sawaal galat hai — dono production-grade hain. Rule of thumb: **simple web app + hosting availability → MySQL**; **complex relations + data integrity + advanced SQL → PostgreSQL**. Aaj kal modern full-stack (Supabase/Neon) zyada PostgreSQL push karta hai, isliye dono aane chahiye — SQL transferable skill hai.

Command quick-difference (side by side)

Kaam	MySQL	PostgreSQL
Connect CLI	`mysql -u root -p`	`psql -U postgres`
Databases list	`SHOW DATABASES;`	`\l`
Use database	`USE school;`	`\c school`

Tables list	`SHOW TABLES;`	`\dt`
Describe table	`DESC students;`	`\d students`
Transaction start	`START TRANSACTION;`	`BEGIN;` (START TRANSACTION bhi chalta hai)
Insert-ignore	`INSERT IGNORE ...`	`INSERT ... ON CONFLICT DO NOTHING`
Backup	`mysqldump`	`pg_dump`
Default port	3306	5432
Config file	`my.cnf` / `my.ini`	`postgresql.conf`

Part 5 — Practical Tasks: Basic → Advance

Rules (trainer taraf se): Har task pehle *bina dekhe* khud likho. Phir hint/mismatch ho to answer dekho. Har task **dono databases mein** chalao — same task, do alag CLI. Yahi "easy way" hai.

Level 1 — Basic (Week 1) 10 tasks

T1. Apni database banao jiska naam `school` ho, use select karo, aur list se confirm karo ke bani.

Hint: MySQL → `CREATE DATABASE school; USE school; SHOW DATABASES;` · PG → `CREATE DATABASE school;` (phir CLI se `psql -d school`) aur `\l`

T2. `students` table banao jisme ho: auto-increment id, name (NOT NULL), email (UNIQUE), age, fee (decimal), active flag, aur created timestamp.

Hint: MySQL → `AUTO_INCREMENT` + `DATETIME DEFAULT CURRENT_TIMESTAMP` · PG → `SERIAL` + `TIMESTAMP DEFAULT NOW()` + `BOOLEAN`

T3. Us table mein **5 students** insert karo (ek INSERT statement mein 3, alag se 2).

Multi-row INSERT dono mein same: `INSERT INTO students (name,email,age) VALUES (...),(...),(...);`

T4. Sirf un students ka naam aur email nikalo jo 18 se bara hain, naam ke hisaab se A→Z.

`SELECT name, email FROM students WHERE age > 18 ORDER BY name ASC;`

T5. Un dhoondo jinka email `gmail` par khatam hota hai aur unki tadaad batao.

`SELECT COUNT(*) FROM students WHERE email LIKE '%gmail';`

T6. Har student ka fee 10% barhao — sirf unka jo 1000 se kam hai. (Pehle SELECT se check, phir UPDATE.)

`SELECT * FROM students WHERE fee < 1000;` → `UPDATE students SET fee = fee * 1.10 WHERE fee < 1000;` — **Warning:** UPDATE/DELETE kabhi WHERE ke bina mat chalao!

T7. Ek student delete karo jiska id 5 hai, aur confirm karo ke delete hua (COUNT pehle vs baad).

`SELECT COUNT(*) FROM students;` → `DELETE FROM students WHERE id = 5;` → dobara COUNT

T8. Ek nayi `departments` table banao (id, title) aur 3 departments insert karo.

Simple DDL + INSERT. PK hona chahiye.

T9. `students` table mein `dept_id` column add karo jo foreign key ho `departments.id` ka — phir 2 students ko department do.

MySQL: `ALTER TABLE students ADD COLUMN dept_id INT REFERENCES departments(id);` (MySQL mein FK ke liye engine InnoDB chahiye) · PG: same syntax chalta hai.

T10. INNER JOIN aur LEFT JOIN dono se student + uska department name nikalo — fark observe karo (jin ka dept NULL hai LEFT mein dikhenge, INNER mein nahi).

`SELECT s.name, d.title FROM students s LEFT JOIN departments d ON d.id = s.dept_id;` — INNER ke liye LEFT ki jagah INNER likho.

Level 2 — Intermediate (Week 2) 10 tasks

T11. 3-table schema design karo (e-commerce): `customers`, `products`, `orders` + `order_items` (junction table). PK/FK sahi lagao.

Order mein ek se zyada product → many-to-many → `order_items (order_id, product_id, qty, price)` alag table.

T12. Department-wise students ki count aur average fee nikalo — sirf woh departments jin mein 5+ students hon.

`SELECT dept_id, COUNT(*), AVG(fee) FROM students GROUP BY dept_id HAVING COUNT(*) >= 5;`

T13. Subquery se un students ko nikalo jo apne department ke average se zyada fee dete hain.

Correlated subquery: `WHERE fee > (SELECT AVG(fee) FROM students s2 WHERE s2.dept_id = s.dept_id)`

T14. Har order ka total amount nikaalo (qty × price ka sum), orders ke saath JOIN karke — GROUP BY order_id.

`SELECT o.id, SUM(oi.qty * oi.price) AS total FROM orders o JOIN order_items oi ON oi.order_id = o.id GROUP BY o.id;`

T15. Ek `view` banao `active_students` jo sirf active students dikhaye — phir usse SELECT karo aur ek column baad mein add karo (`ALTER VIEW`).

MySQL: `CREATE VIEW ...` / `ALTER VIEW ...` · PG: same ya `CREATE OR REPLACE VIEW`

T16. Email column par index banao, phir `EXPLAIN` se pehle/baad ka query plan compare karo.

MySQL: `EXPLAIN SELECT * FROM students WHERE email='x';` · PG: `EXPLAIN ANALYZE SELECT ...;` — type: ALL (full scan) vs index (ref) dekho.

T17. Transaction ka demo: ek accounts table banao, 2 rows mein balance, phir ₹1000 transfer karo — beech mein jaan bujh kar error karao aur `ROLLBACK` karo, confirm karo ke balance same hai.

MySQL: `START TRANSACTION; ... COMMIT;/ROLLBACK;` · PG: `BEGIN; ... COMMIT;/ROLLBACK;`

T18. Upsert karo: agar email pehle se hai to update warna insert. (MySQL: `ON DUPLICATE KEY UPDATE` · PG: `ON CONFLICT (email) DO UPDATE`) — dono mein chalao aur fark dekho.

T19. Har student ka email naam aur domain ke hisaab se nikalo: `first part` aur `@ke baad wala part` alag columns mein.

MySQL: `SUBSTRING_INDEX(email,'@',1)` / `SUBSTRING_INDEX(email,'@',-1)` · PG: `SPLIT_PART(email,'@',1)` / `SPLIT_PART(email,'@',2)` — **yahi difference hai jo cheat sheet se yaad karna hai**.

T20. Date ka kaam: sirf aaj ke created students, aur har student ka "member for X days" nikalo.

MySQL: `WHERE DATE(created_at) = CURDATE()` · `DATEDIFF(NOW(), created_at)` · PG: `WHERE created_at::DATE = CURRENT_DATE` (ya `CURRENT_DATE`) · `AGE(created_at)::INTEGER` / `NOW() - created_at`

Level 3 — Advance (Week 3–4) 10 tasks

T21. Window function: har department ke andar sabse fee ka rank karo — bina GROUP BY ke row ke saath rank column chahiye.

`SELECT name, dept_id, fee, RANK() OVER (PARTITION BY dept_id ORDER BY fee DESC) AS rnk FROM students;` (dono DB mein — MySQL 8+ chahiye)

T22. CTE (WITH clause): department-wise total aur us department ka percentage of grand total — ek hi query mein.

`WITH dept_tot AS (SELECT dept_id, SUM(fee) t FROM students GROUP BY dept_id) SELECT dept_id, t, ROUND(t*100.0/(SELECT SUM(t) FROM dept_tot),2) pct FROM dept_tot;`

T23. Self join: `employees` table mein `manager_id` hai — har employee ka manager ka naam LEFT JOIN se nikalo.

`SELECT e.name, m.name AS manager FROM employees e LEFT JOIN employees m ON m.id = e.manager_id;`

T24. Advanced JOIN: un customers ko nikalo jne kabhi order nahi kiya (LEFT JOIN + IS NULL) — phir wahi EXISTS se likho, do tariko ka EXPLAIN compare karo.

T25. JSON ka kaam: har student ke `meta` JSON column se `meta.city` extract karo aur city par filter lagao.

PG (JSONB): `meta->>'city' = 'Karachi'` + index: `CREATE INDEX ON students ((meta->>'city'));` · MySQL: `JSON_EXTRACT(meta,'$.city') = 'Karachi'` ya shorthand `meta->'$.city'`

T26. Query optimization: ek badi table (1000+ rows banao `INSERT ... SELECT` se) par slow query chalao, `EXPLAIN ANALYZE` parho (seq scan vs index scan), index banao, phir timing compare karo.

Generate rows: `INSERT INTO students (name, fee) SELECT CONCAT('St', seq), random()*5000 FROM generate_series(1,1000) seq;` (PG) · MySQL mein numbers table ya loop use karo.

T27. User permissions: ek naya DB user banao jise sirf SELECT ka haq ho, usse connect karke INSERT try karo (error aani chahiye), phir GRANT karo.

MySQL: `CREATE USER 'app'@'%' IDENTIFIED BY 'x'; GRANT SELECT ON school.* TO 'app'@'%';` · PG: `CREATE ROLE app LOGIN PASSWORD 'x'; GRANT SELECT ON ALL TABLES IN SCHEMA public TO app;` (PG mein roles grouped hote hain)

T28. Backup & restore ka drill: poori database backup lo, ek table drop karo, phir backup se restore karke wapas lao.

MySQL: `mysqldump` / `mysql < backup.sql` · PG: `pg_dump -Fc` / `pg_restore`

T29. Normalization exercise: ek messy single table di hai (customer ka naam, uske multiple courses comma-string mein, uska city, city ka zip) — use 3NF mein todo aur FK lagao.

T30. Capstone: Same e-commerce schema (T11) **MySQL AND PostgreSQL dono mein** banao, 50+ rows insert karo, 5 advanced queries (CTE, window, self-join, view, index+EXPLAIN) chalao — aur dono ke syntax differences ka apna comparison note banao. *Ye kaam complete hone par tum dono databases confidently use kar sakte ho.*

Part 6 — 30-Day Learning Plan (roadmap)

Days	Kya seekhna hai	Kya banao / kaise practice
Day 1–2	Foundation: DB, RDBMS, SQL, tables, PK/FK, ACID	Part 0 parho; oncompiler par 5 SELECT likho bina install ke
Day 3–5	MySQL: connect, DDL, CRUD, WHERE/ORDER/LIMIT	MySQL local install karo; Level 1 tasks T1–T7 karo
Day 6–7	MySQL: JOIN + GROUP BY + basic functions	T8–T10 + mini project: school DB (3 tables)
Day 8–9	PostgreSQL install + psql meta-commands	Wahi school DB phir se PG mein banao (differences note karo)
Day 10–12	PG: SERIAL vs AUTO_INCREMENT, BOOLEAN, ILIKE, views, indexes	Level 1 tasks PG mein + T15, T16
Day 13–14	Differences ka revision + command side-by-side	Part 4 ka table khud likh kar yaad karo; T19, T20 (functions difference)
Day 15–17	Intermediate SQL: subquery, HAVING, transactions, upsert, indexes + EXPLAIN	Level 2 tasks T11–T18 dono DB mein
Day 18–20	E-commerce schema design + relationships	T11 ka full schema + sample data (50 rows) + 5 JOIN queries
Day 21–24	Advanced: window functions, CTE, self join, JSON/JSONB, permissions	Level 3 tasks T21–T27
Day 25–26	Backup/restore + normalization + query tuning	T28, T29; EXPLAIN se 2 slow queries fast karo
Day 27–29	Capstone — same project dono mein	T30: e-commerce in MySQL + PostgreSQL, full comparison notes
Day 30	Revision + interview prep	Neeche wale 10 interview questions khud se jawab likho

Practice rhythm (roz ka nizam)

- **Roz 1–1.5 ghanta:** 30 min concept + 60 min task. Sirf parhne se nahi hoga — **likhne se hoga**.
- **Har cheez dono mein chalao:** Ek hi query MySQL aur PG dono mein likho — difference yaad rehta hai.
- **Ek hi final project:** E-commerce ya school management — MySQL mein banao, phir PG mein replicate karo.
- **Har SELECT ke baad EXPLAIN** dekhna shuru karo — optimizer samajhna advanced skill hai.

Part 7 — Trainer ke Pro Tips + Interview Quick Notes

> **10 zaroori tips:**
>
> 1. Kabhi bhi `UPDATE` / `DELETE` WHERE ke bina mat chalao — pehle SELECT se check karo.
> 2. Har table mein `id` (PK), `created_at`, zaroorat ho to `updated_at` rakho.
> 3. Foreign keys lagao — database level par integrity rakhogi to app mein kam bugs.
> 4. Sab kuch ek table mein mat dalo — 3NF follow karo, phir zaroorat par judo (JOIN se).
> 5. Index magic nahi hai — sirf WHERE/ORDER BY/JOIN wale columns par lagao; har column par mat lagao.
> 6. Money ke liye FLOAT nahi — MySQL `DECIMAL` / PG `NUMERIC` use karo.
> 7. IDs hamesha ORDER BY mein do — `LIMIT` bina ORDER BY ke non-deterministic hai.
> 8. Transaction ek complete business step ho — bank transfer jaisa: ya dono updates ya koi nahi.
> 9. Backup automation pehle hi din se — `mysqldump` / `pg_dump` cron mein.
> 10. PostgreSQL mein `JSONB` powerful hai lekin relational data ko JSON mein mat chhapo — har cheez ka apna jagah.

Interview ke 10 sawaal (khud jawab do pehle)

Sawaal	Ek line answer
ACID kya hai?	Transaction ki 4 guarantees: Atomicity, Consistency, Isolation, Durability
PRIMARY KEY vs UNIQUE?	PK: uniquely identify + NOT NULL + ek table mein ek; UNIQUE: multiple allowed + NULL allowed (usually)
INNER JOIN vs LEFT JOIN?	INNER: sirf dono mein match; LEFT: left table sab rows, match na ho to NULL
WHERE vs HAVING?	WHERE rows par filter (GROUP BY se pehle); HAVING groups par filter (GROUP BY ke baad)
MySQL aur PostgreSQL ka farq?	Part 4 ka summary: speed/simplicity vs power/standards; AUTO_INCREMENT vs SERIAL; JSON vs JSONB; backtick vs quotes
SERIAL kya hai?	PG ka auto-increment — sequence bana kar id khud set karta hai (naya tareeqa: IDENTITY)
JSONB kyun tez hai?	Binary format — parse nahi karna parta, GIN index lag sakti hai, operators (`@>`) fast hain
Index kya hai, nuksan kya?	Lookup pointer — SELECT fast, lekin INSERT/UPDATE/DELETE slow + memory
Normalization kya hai?	Data redundancy hatana (1NF→2NF→3NF) — same data ek hi jagah
Subquery vs JOIN?	Dono se kaam hota hai; kabhi kabhi JOIN behtar, kabhi subquery — EXPLAIN se check karo

---